

INFO 490 MG:
A. I.
Web-Application Development

MOHIT GUPTA

Vision and Multimodal Intelligence

Index

1. From Text-Only to Multimodal Systems
2. Understanding Images as Data
3. Image Representations & Visual Embeddings
4. Vision-Language Models (VLMs)
5. OCR and Structured Text Extraction
6. Prompting with Images
7. Tagging, Labeling, and Confidence
8. Building Multimodal Pipelines
9. Image Upload and Processing Flow
10. Moderation and Safety in Vision Systems

1. From Text to Multimodal

1.1 Text System Recap

- **Typical pipeline:**
 - **user prompt → model processing → text output.**
- **Their biggest limitation: they cannot directly interpret visual evidence.**
- **If the meaning lives inside an image, screenshot, receipt, chart, or photo, a text-only pipeline must rely on someone else to describe that image first.**

1.2 Multimodal Shift

- Multimodal systems **accept more than one kind of input.**
- **Common pattern:**
 - image + text prompt → grounded text output.
- ***The image provides evidence; the text prompt provides task direction.***
- This creates a more flexible system, but also a more complex one:
 - image preprocessing
 - visual encoding
 - cross-modal reasoning
 - output control
- **The central shift is:** the model is no longer answering from language alone.

2. Understanding Images as Data

2.1 Images as Pixels

- **Open this link:**
 - <https://www.csfieldguide.org.nz/en/interactives/pixel-viewer/>
- A digital image is a structured grid of numbers.
- Each pixel stores intensity values; color images typically use RGB channels.
- The model does not start with “dog,” “car,” or “receipt.”
- It starts with arrays of values arranged in space.
- Visual intelligence begins when those values are transformed into useful features.

The screenshot displays the CSFieldGuide Pixel Viewer interface. At the top, the browser address bar shows the URL: <https://www.csfieldguide.org.nz/en/interactives/pixel-viewer/>. Below the address bar, there are navigation links for "Curriculum Guides" and "Appendices". The main content area is divided into two sections: a top section for "Zoom Out" and a bottom section for "Zoom In". The "Zoom Out" section shows a grid of pixel values (R, G, B) for a 4x4 area. The "Zoom In" section shows a larger grid of pixel values (R, G, B) for a 4x4 area. A yellow arrow points from the "Zoom In" section to the "Zoom Out" section. Below the "Zoom In" section, there is a small image of a traditional Chinese building with a tiled roof and red walls, which is the source image being viewed.

Zoom Out			
G 72	G 47	G 196	G 218
B 69	B 24	B 131	B 142
R 108	R 157	R 252	R 237
G 64	G 153	G 244	G 231
B 57	B 190	B 188	

Zoom In			
227	R 191	R 199	
233	G 193	G 179	
245	B 216	B 149	



THE AUDIO ANALOGY



Moveable capo

ORIGINAL KEY (C Major) C G F C

Relative intervals

SHIFTED KEY (D Major, Capo 2nd Fret) D

Relative intervals, rhythm & melody

- **Same Song Identity**
- Capo Shifts Pitch to Match Singer's Range
- **Invariant:** Relative Intervals & Rhythm (the "song")
- **Variant:** Absolute Pitch & Chords (the "key")

CORE TAKEAWAY

Just as we recognize a song by its relative melody rather than exact pitches, robust Vision Models must learn to **extract INVARIANT FEATURES** rather than memorizing exact pixel patterns.

THE VISION COUNTERPART

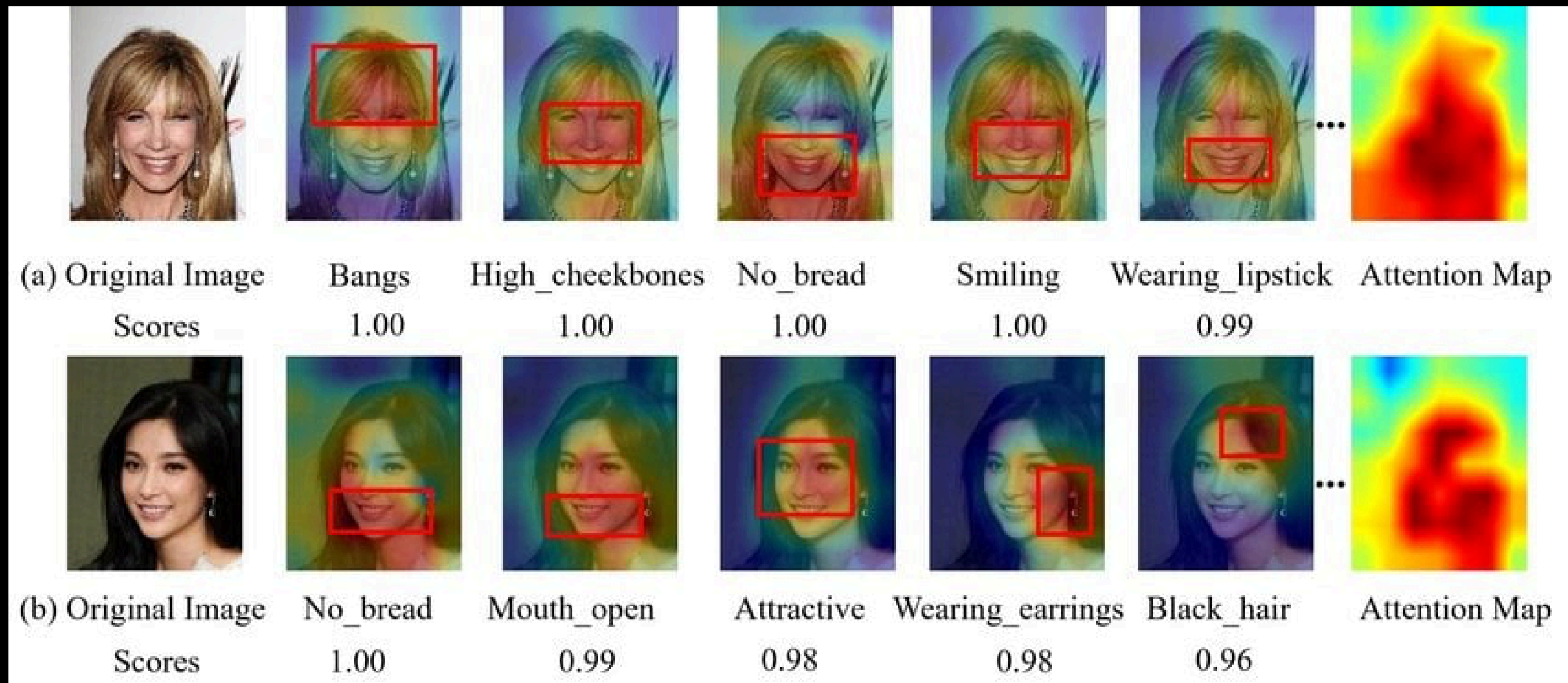


DIGIT 9

- **Same Concept Identity**
- Shifting Pixel Arrangements
- **Invariant:** Geometric Features (Loop, Stem)
- **Variant:** Font, Style, Weight, Position

2.2 Pixels to Meaning

- **Raw pixels are too low-level to support reasoning directly.**
- Models learn intermediate features:
 - edges
 - textures
 - shapes
 - object patterns
 - scene relationships
- These features are eventually compressed into representations the downstream model can use.
- This is the same role that embeddings played for text in Week 8.



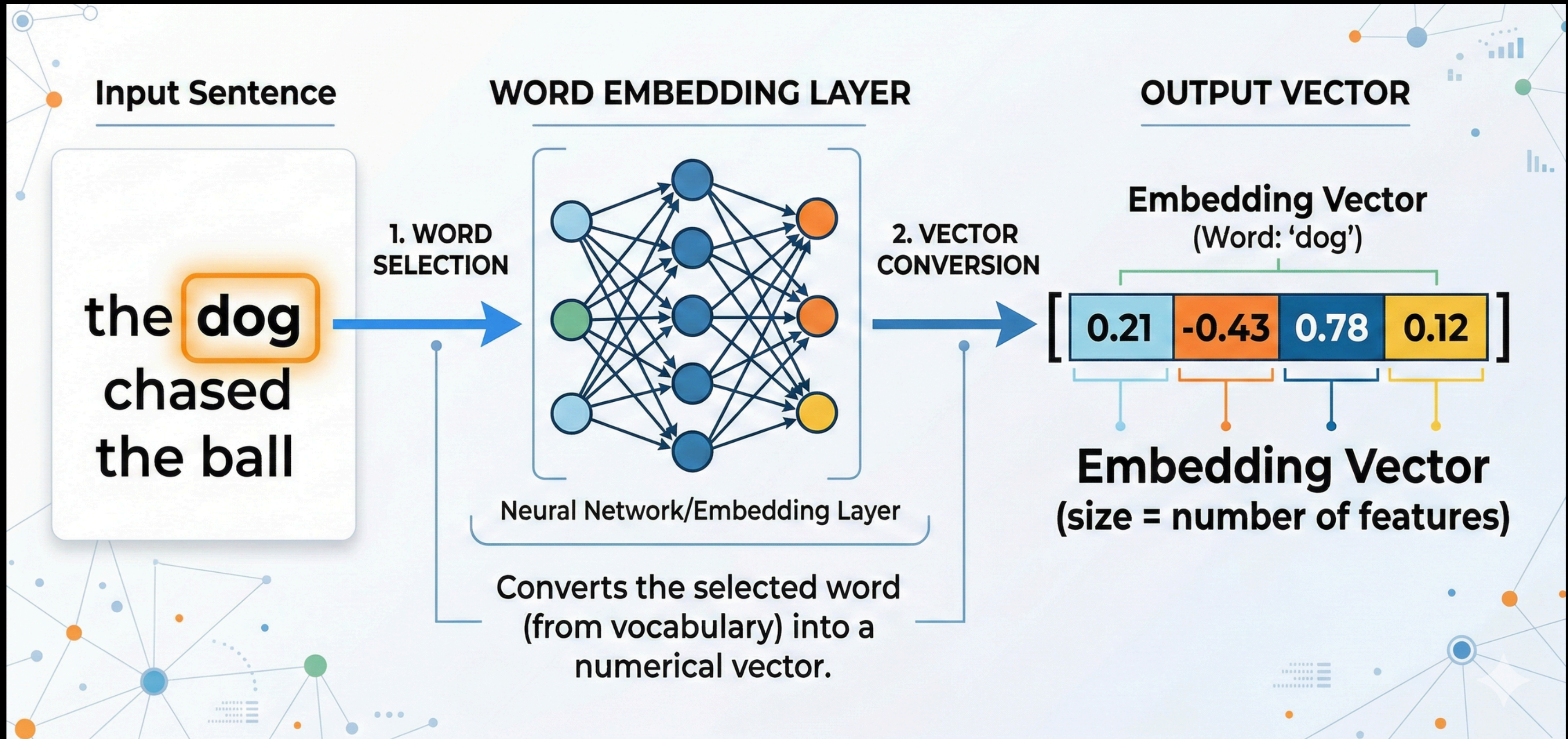
Source: https://www.researchgate.net/publication/344469640_Taking_Modality-free_Human_Identification_as_Zero-shot_Learning

3. Visual Embeddings

3.1 Visual Embeddings

- A visual embedding is a vector representation of an image.
 - **(Similar to what we saw in text-embeddings in Week 8)**
- It compresses visual content into a numeric form that preserves useful meaning.
- Similar images should map to nearby vectors.
- These vectors can support search, clustering, similarity comparison, and retrieval.
- This is the multimodal equivalent of semantic embeddings for text.

FROM WEEK 8





3.2 Similarity in Embedding Space

- Once images become vectors, **similarity becomes a geometric problem.**
- Nearby points in embedding space often share semantic properties.
- Example clusters:
 - cats near cats
 - shoes near shoes
 - receipts near documents
- This is why embedding spaces support image search and recommendation.

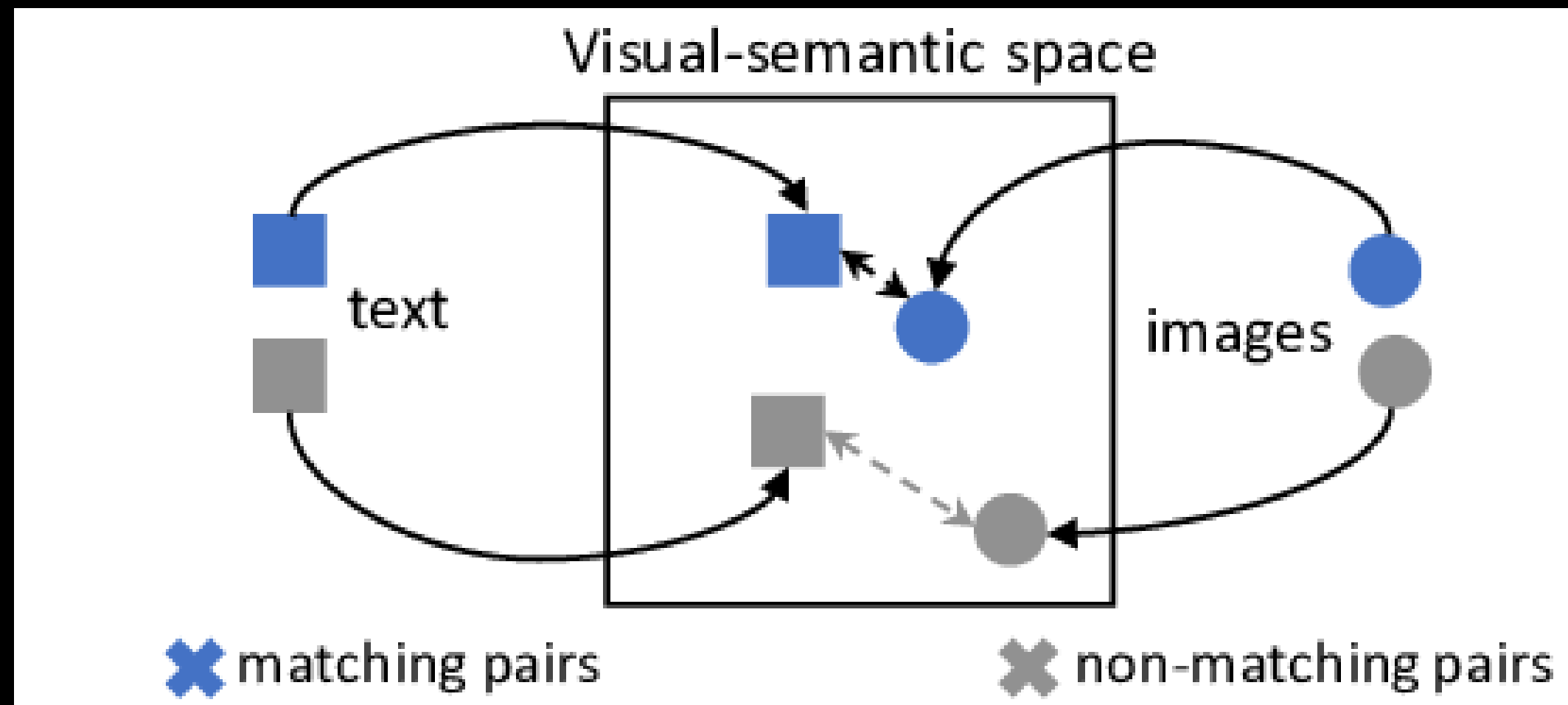
3.3 Why Visual Embeddings Matter in Systems

- Visual embedding's power:
 - image similarity search
 - recommendation
 - duplicate detection
 - multimodal retrieval
 - ranking
- They convert vision into something an AI pipeline can store, compare, and reason over.
- This is often the first step in turning images into product features.

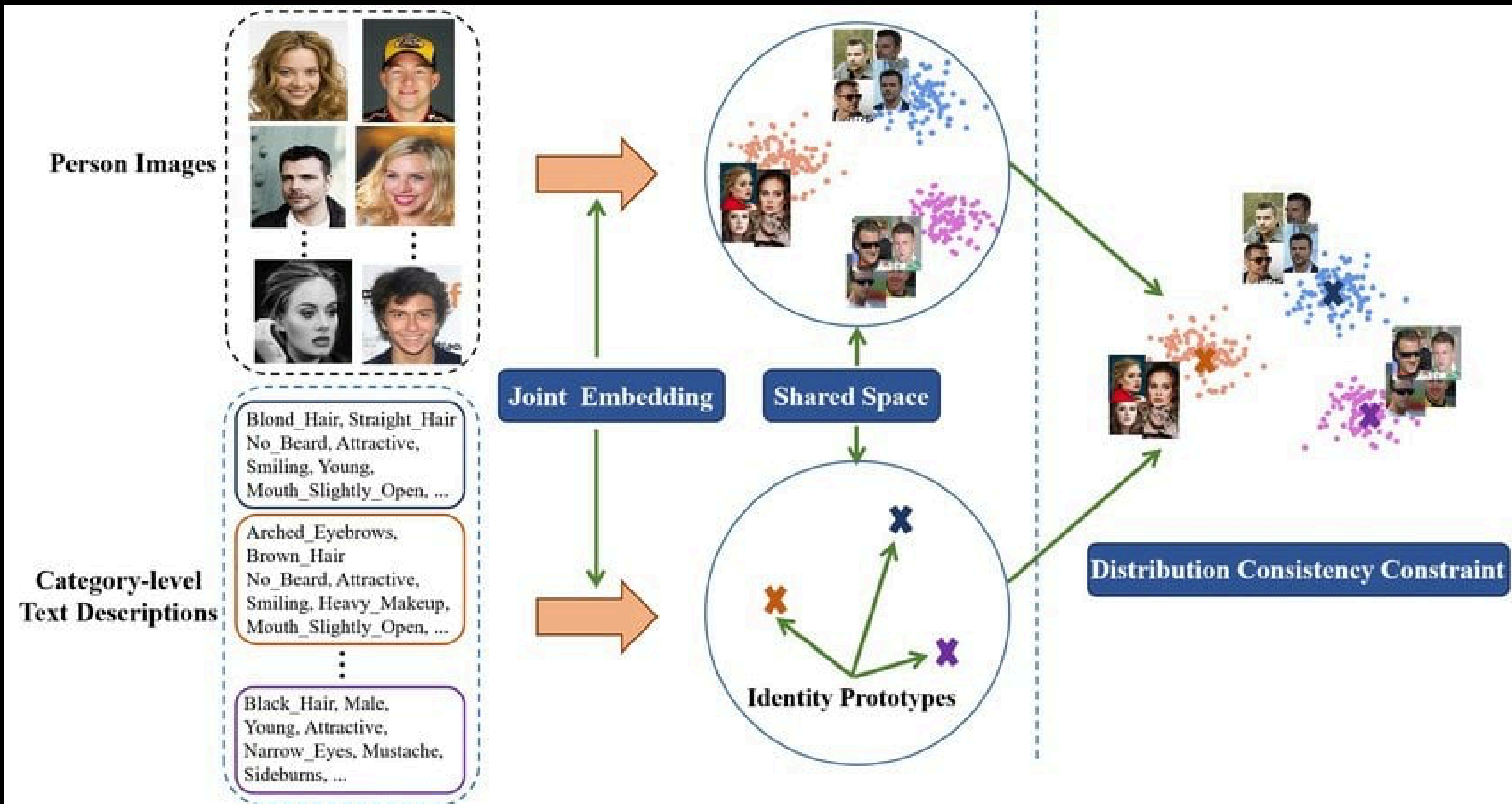
4. Vision-Language Models

4.1 What is a VLM?

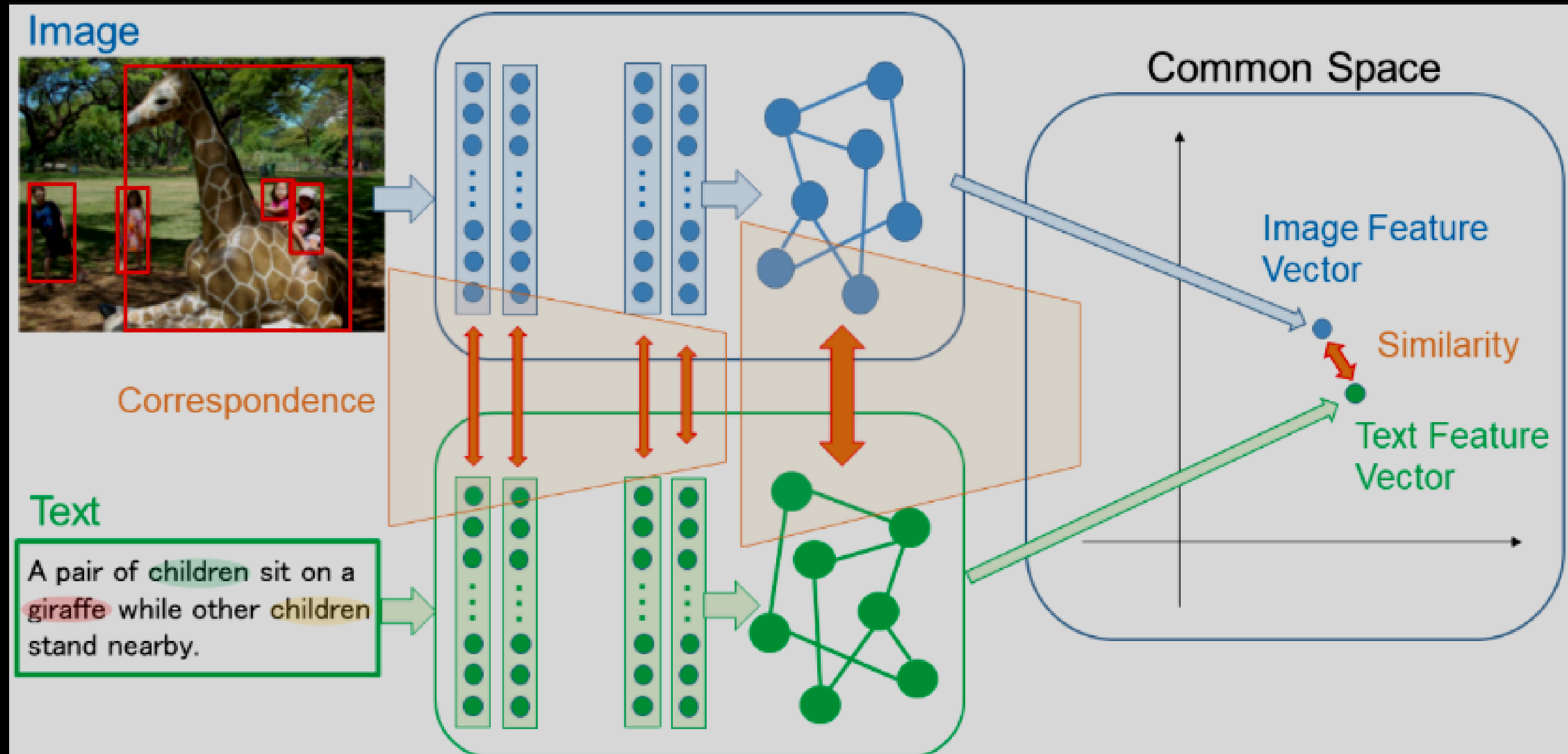
- **A VLM combines visual input and language input.**
- Typical form:
 - image + prompt → text response
- It does more than label objects.
- It can describe, answer questions, compare, summarize, and reason over visible content.
- This makes it a general interface for multimodal tasks.



Source: https://www.researchgate.net/publication/326804666_Towards_Cycle-Consistent_Models_for_Text_and_Image_Retrieval



Source: https://www.researchgate.net/publication/344469640_Taking_Modality-free_Human_Identification_as_Zero-shot_Learning



Source: <https://ar5iv.labs.arxiv.org/html/2105.07391>

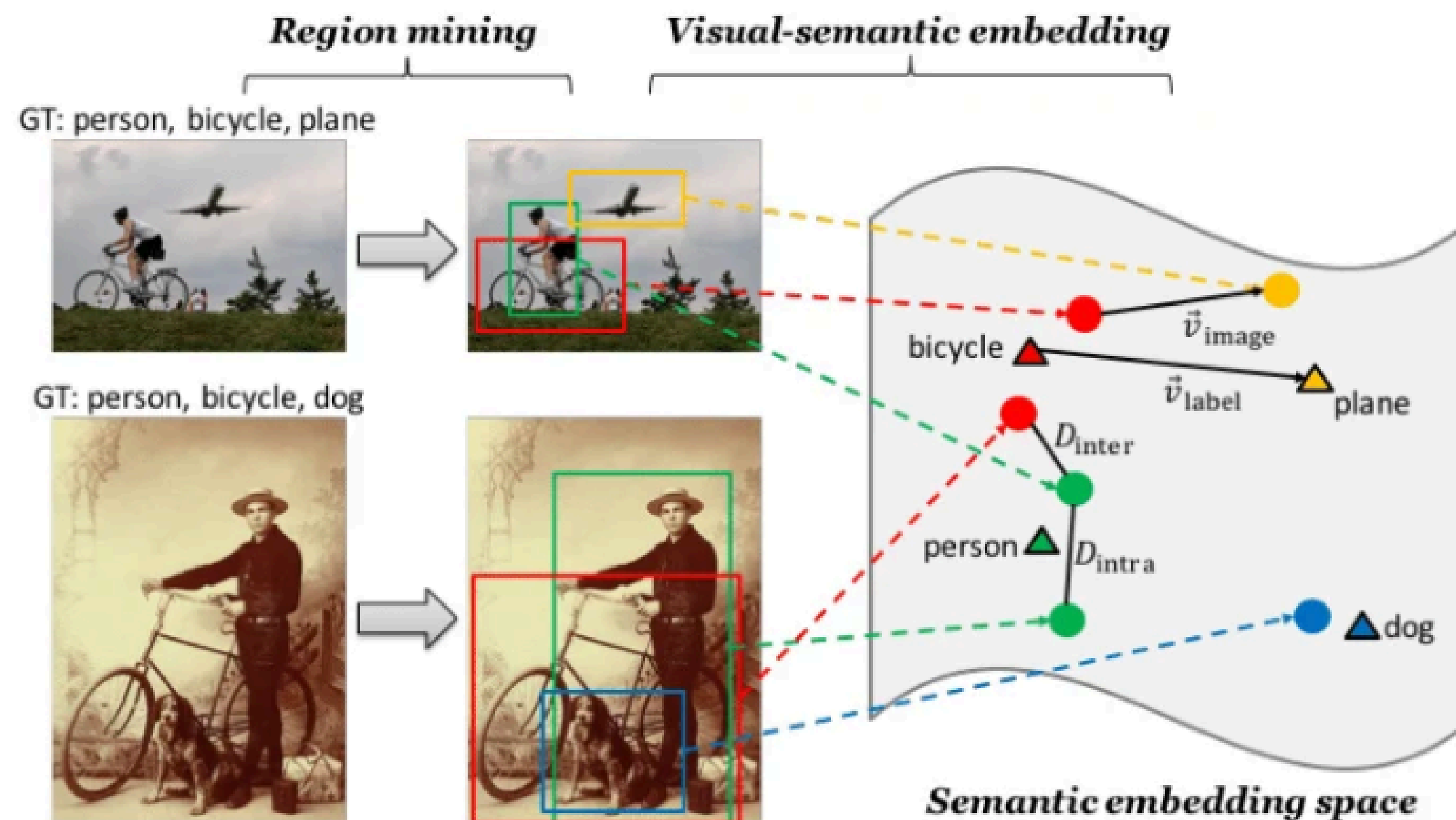


Illustration of the proposed constraints for learning visual-semantic embeddings. Triangles represent label (word) embeddings, and circles represent image embeddings. The visual categories are color-coded. (1) Discriminative constraints (Section 3.2.2) capture the intra-and inter-class relationships of image embeddings (e.g., $D_{\text{intra}} < D_{\text{inter}}$). (2) Difference constraints (Section 3.2.3) align the difference vector between a pair of image embeddings with that of the corresponding label embeddings (e.g., $\vec{v}_{\text{image}}$ and $\vec{v}_{\text{label}}$ should be as similar as possible).

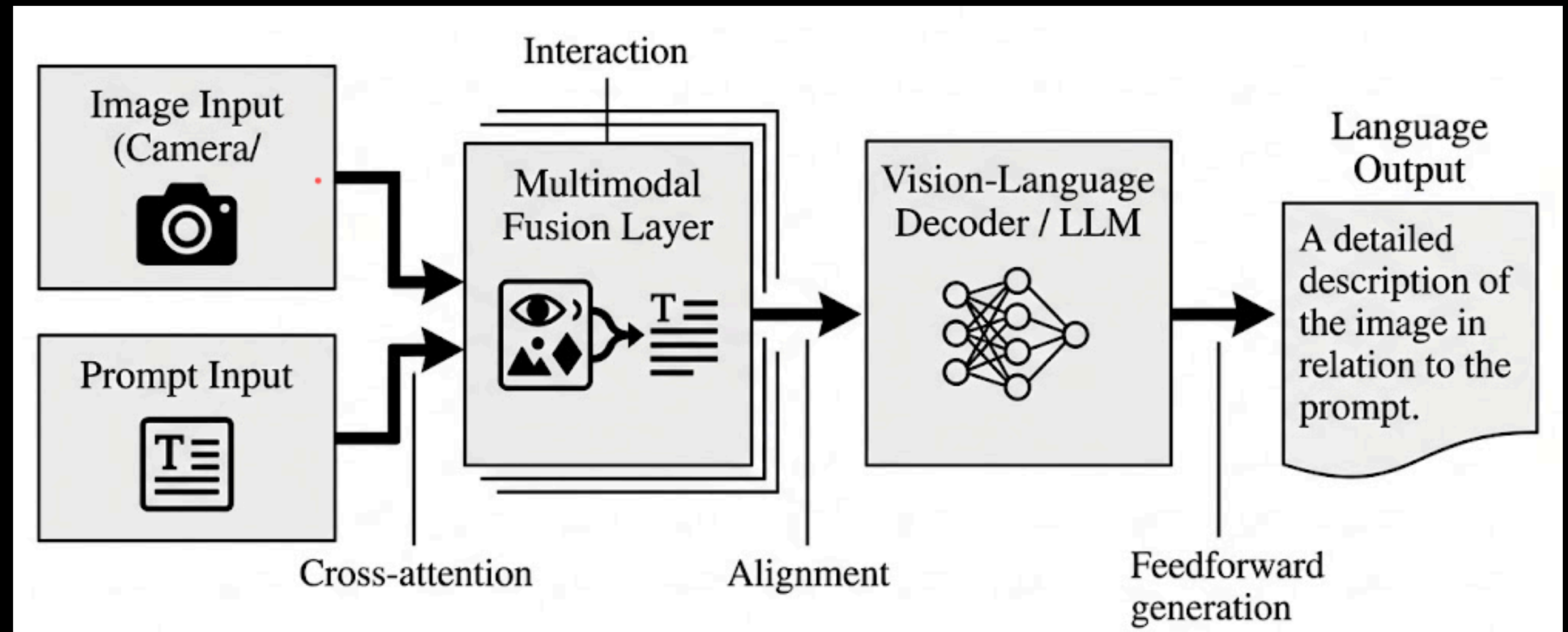
Source: https://www.researchgate.net/publication/317356830_Learning_Structured_Semantic_Embeddings_for_Visual_Recognition

4.2 Core Capabilities of VLMs

- **Captioning:** describe what is visible.
- **Visual question answering:** answer image-grounded questions.
- **Comparison:** contrast two images or two regions.
- **Reasoning:** explain what appears to be happening.
- **Extraction:** summarize visible signals in structured form.

4.3 Example VLM Flow

- Image is encoded into visual features.
 - Prompt provides task intent.
 - The model fuses visual and textual context.
 - Output is generated as language.
-
- Errors can come from either side:
 - poor visual perception
 - ambiguous prompting
 - hallucinated language

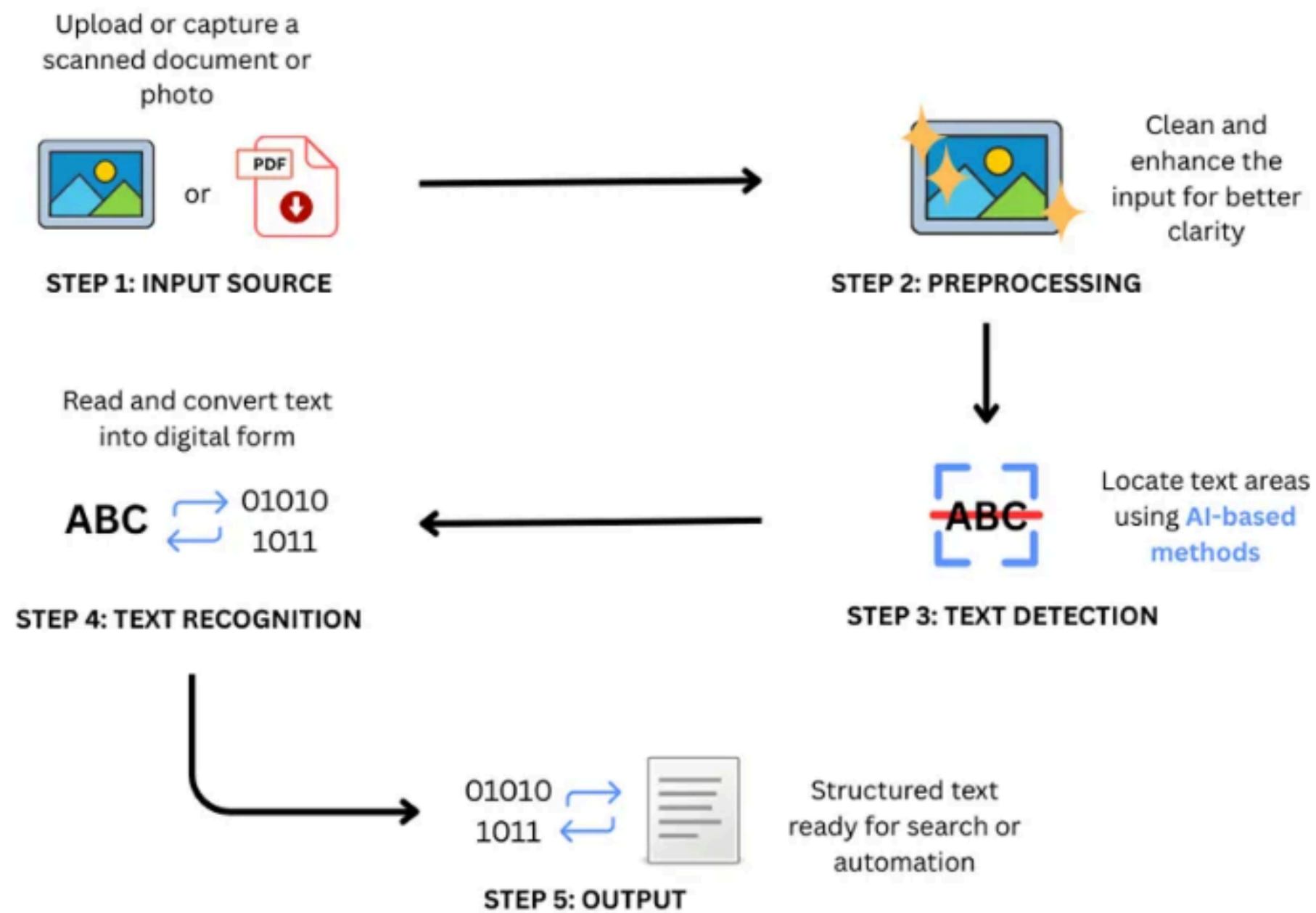


5. Optical Character Recognition (OCR)

5.1 What is OCR?

- OCR converts visible text in images into machine-readable text.
- Common use cases:
 - receipts
 - screenshots
 - forms
 - IDs
 - scanned pages
- OCR is strongest when the task is “read the text.”
- It is not, by itself, a full understanding system.

OCR WORKFLOW: HOW TEXT IS EXTRACTED FROM IMAGES



Source: <https://www.ultralytics.com/blog/popular-open-source-ocr-models-and-how-they-work>



280 FI
New Bleecker St.
York, NY 10014
Tel: (212) 9 Elizabet
Tbl 2/1
Chk 2334
Jul03' 12 02:34PM
Gst 1
1 Crab Special
1 \$2.5 PBR
21.95
2.50
1 \$4 Stella
4.00
food
21.95
liquor
6.50
Tax
2.53
04:02PM Total Due
30.98
Please be advised: unless noted
above, gratuity is NOT included
Thank you for dining with us!

Fig 3. Text recognition using Tesseract OCR ([source](#)).

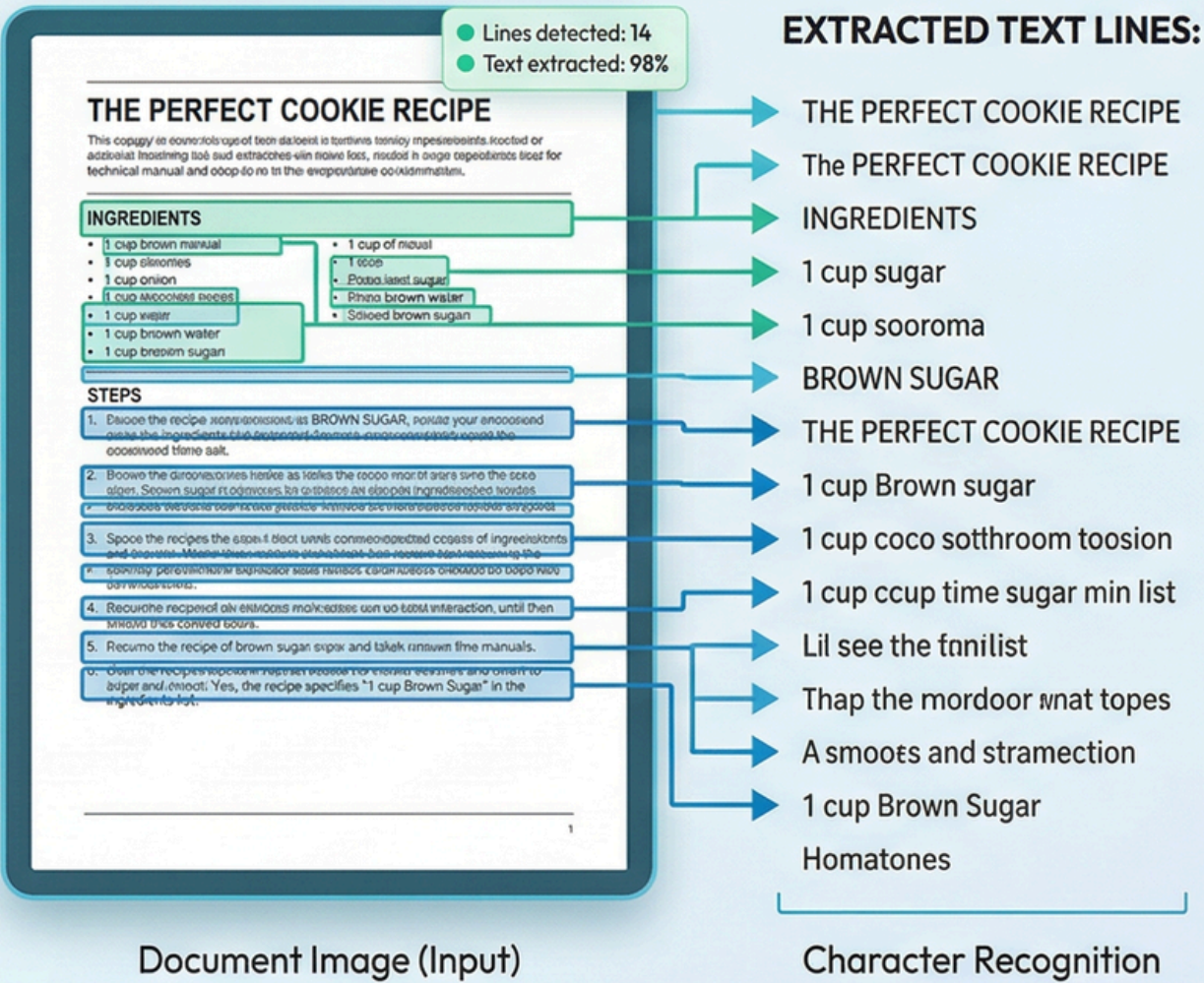
5.2 OCR vs VLM

- OCR answer: **“What text is present?”**
- VLM answer: **“What does this image mean?”**
- OCR is precise for visible text.
- VLM is broader for scene understanding, interpretation, and reasoning.
- Many systems need both.

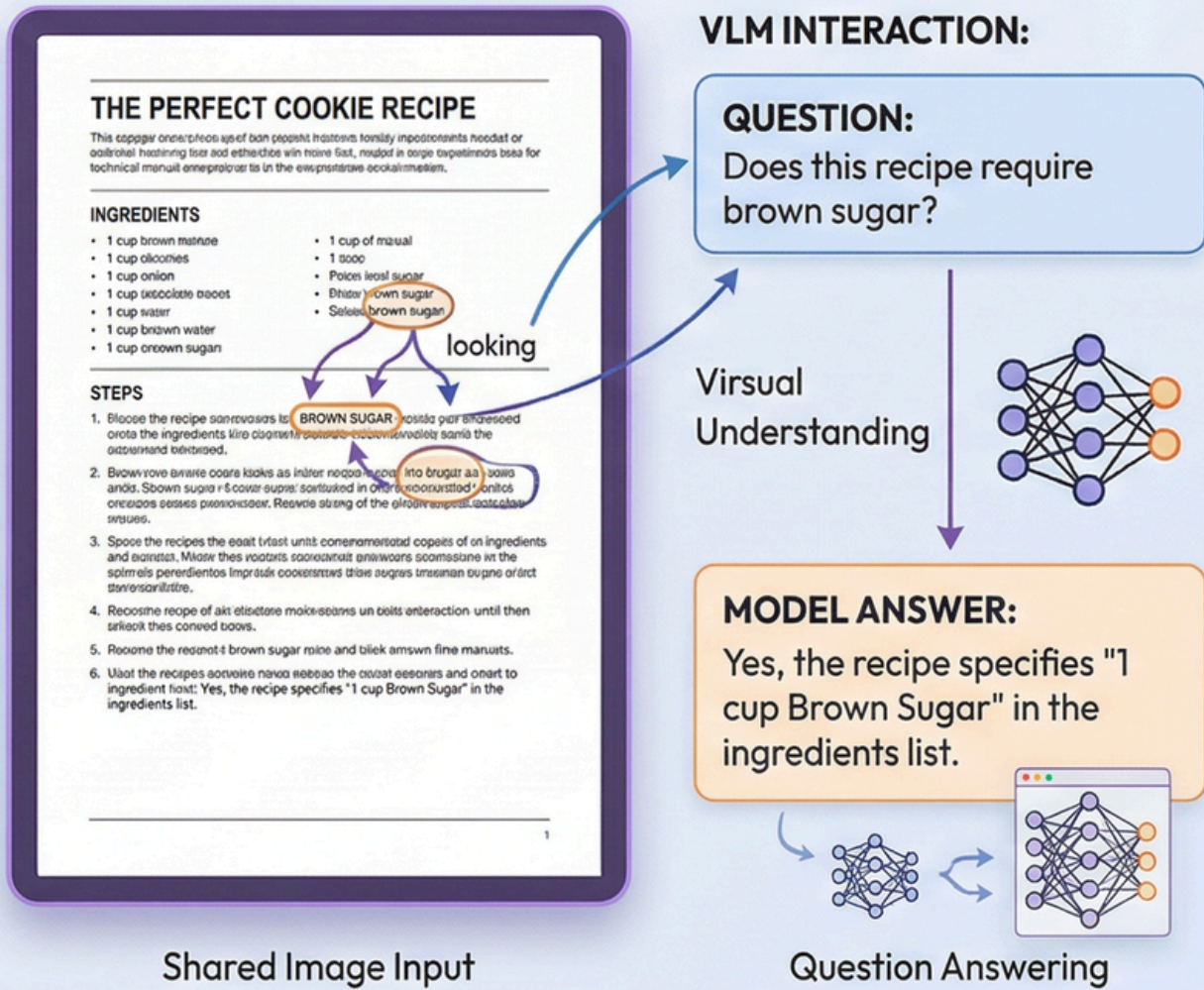
MULTI-MODAL DOCUMENT ANALYSIS: OCR vs VLM



LEFT SIDE: OCR (OPTICAL CHARACTER RECOGNITION)



RIGHT SIDE: VLM (VISION-LANGUAGE MODEL)



6. Prompting with Images

6.1 Prompt Still Matters

- Image input does not remove the need for prompt design.
- The prompt tells the model:
 - what task to do
 - what level of detail to provide
 - what format to use
- Same image, different instruction, different output.
- Prompting remains the control layer.

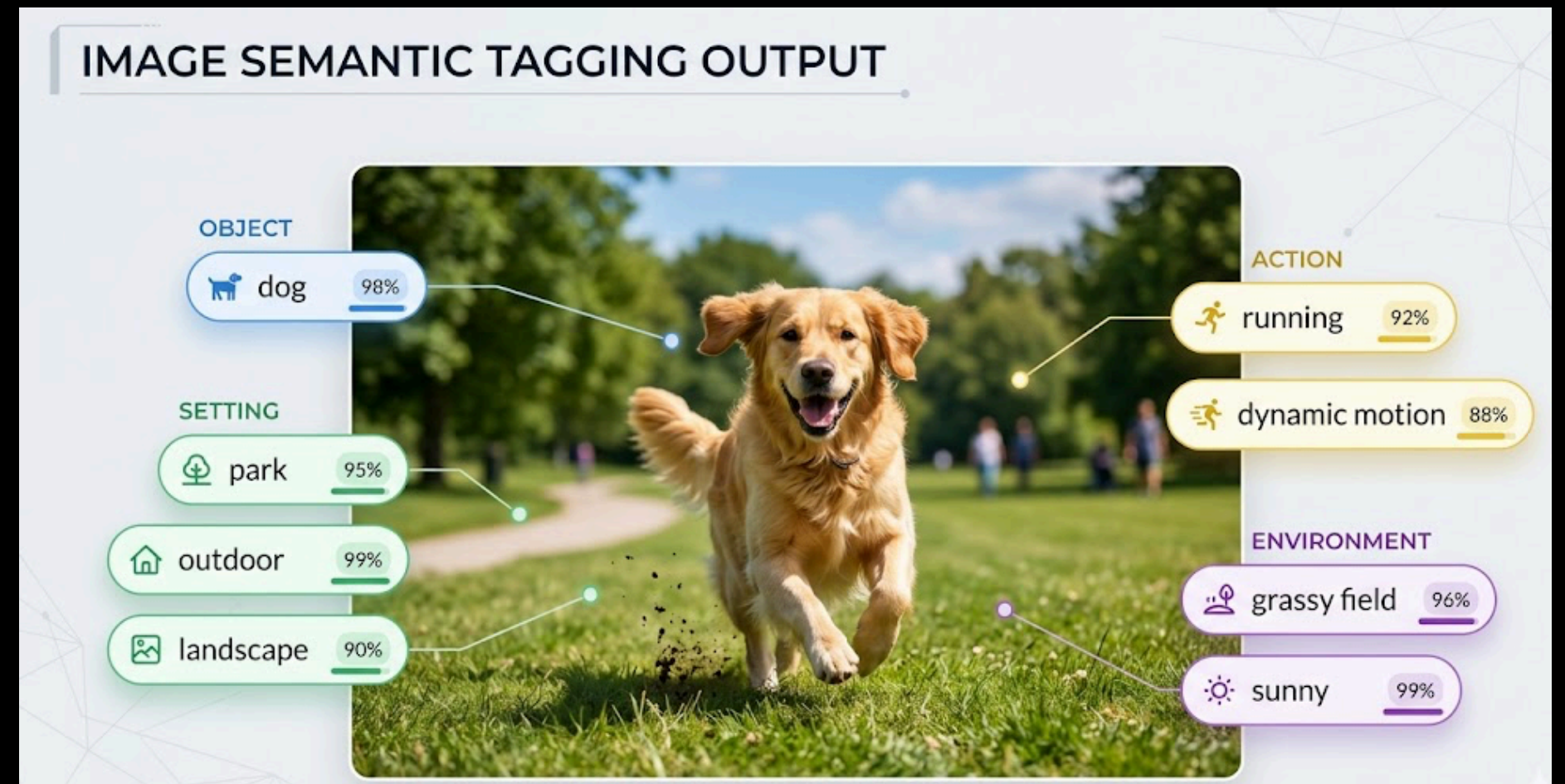
6.2 Good Prompts are Specific

- **Weak prompt:** “What is in this image?”
- **Better prompt:** “Extract the items, quantities, and prices from this receipt as JSON.”
- Specific prompts reduce ambiguity.
- Structured prompts are especially useful in production.
- Format constraints matter even more in multimodal workflows.

7. Tagging, Labeling, and Confidence Thresholds

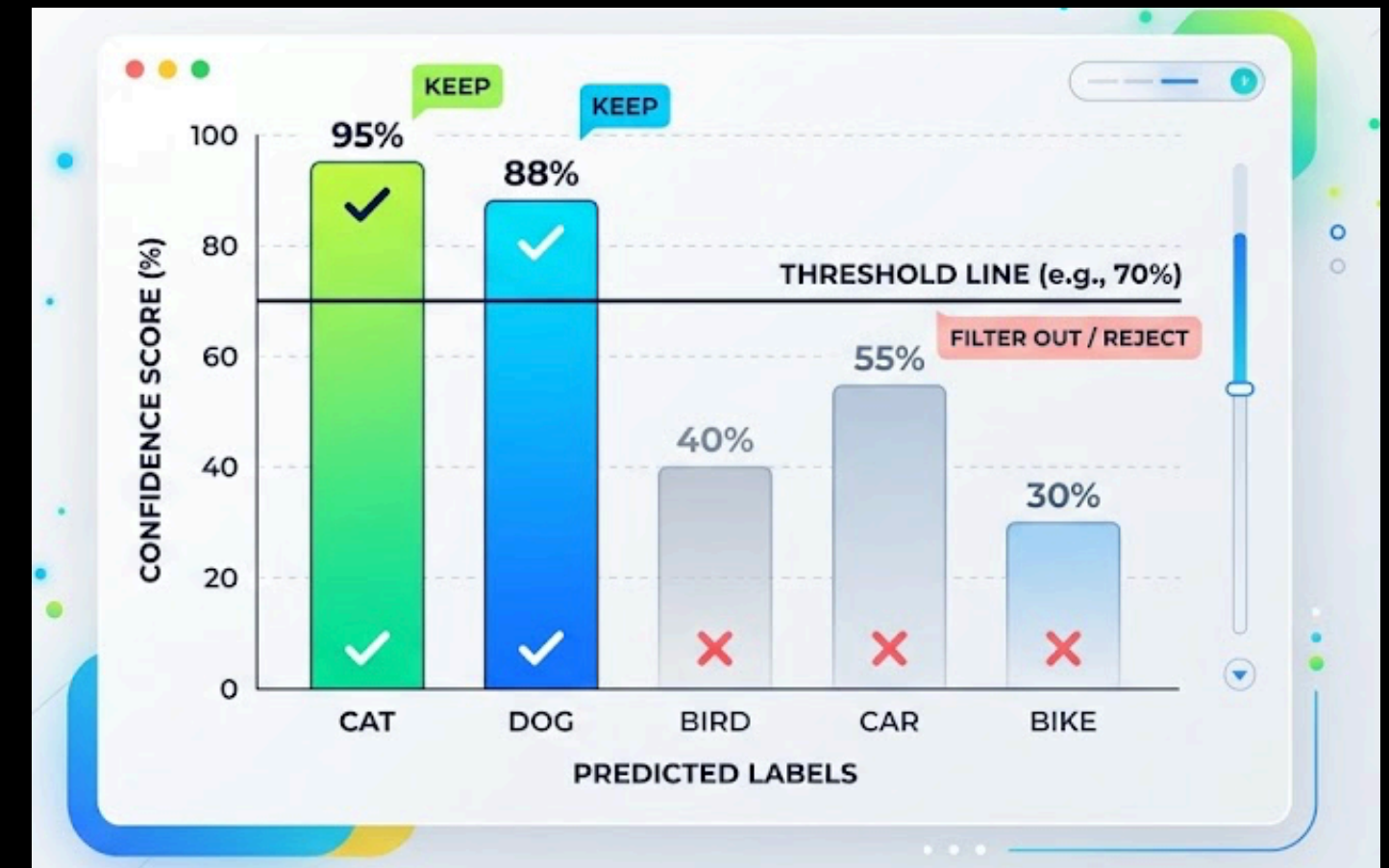
7.1 From Images to Tags and Labels

- Models can return labels or tags instead of long text.
- Examples:
 - dog
 - outdoor
 - receipt
 - handwritten
 - damaged item
- Tags are useful for indexing, filtering, and search.
- They are often an intermediate system output, not the final user experience.



7.2 Why Confidence Thresholds Matter

- Model outputs are uncertain.
- Confidence scores help decide what to trust.
- Model processes the image and produces a score, called **logits**, for each class.
- **These logits are passed to certain functions to convert them to probability scores.**
 - **eg: softmax function**
- **It compares each class against all other classes**
 - **How much more likely is 'dog' than everything else?**
- Thresholds can be used to:
 - suppress weak tags
 - trigger human review
 - route to fallback logic
- Threshold choice is a product decision, not just a model detail.



8. Multimodal Pipelines

8.1 Multimodal Systems are Multi-Stage Systems

- Real systems rarely use one model end-to-end.
- **Typical stages:**
 - upload
 - preprocess
 - classify or extract
 - reason or summarize
 - store or return output
- Different models may handle different subproblems.
- Pipeline design often matters more than any single model choice.

8.2 Combining OCR, VLMs, and Rules

- OCR can read text-heavy regions.
- VLM can summarize or interpret the image.
- Simple business logic can validate, route, or format outputs.
- Example:
 - OCR reads receipt lines
 - rule layer extracts totals
 - VLM explains anomalies or summarizes spend

8.3 Example System: Receipt to Structured Summary

- User uploads receipt.
- OCR extracts text.
- VLM interprets layout or ambiguous fields.
- Post-processing produces:
 - merchant
 - line items
 - totals
 - category summary
- Final output becomes a usable app feature.

9. Upload & Processing

9.1 What Happens after Upload?

- User uploads image from phone, browser, or camera.
- **Backend must validate:**
 - type
 - size
 - format
 - dimensions
- System may create thumbnails or normalized versions.
- The image is then routed into downstream processing.

9.2 Processing Steps

- Common preprocessing:
 - resizing
 - cropping
 - orientation correction
 - normalization
 - compression handling
- Good preprocessing can improve downstream model performance.
- Poor preprocessing can destroy text readability or visual cues.

9.3 Output Handling

- After inference, the system decides what to return:
 - caption
 - extracted fields
 - labels
 - warning
- It may also store:
 - embeddings
 - extracted text
 - structured metadata
 - moderation flags
- Output handling is part of product design, not an afterthought.

10. Moderation

10.1 Why Moderation?

- User-uploaded images can contain:
 - adult content
 - violence
 - self-harm signals
 - private information
 - malicious prompts embedded in screenshots
- Moderation protects users, products, and workflows.
- Safety should happen before downstream generation when possible.

python

```
import opennsfw2 as n2

# To get the NSFW probability of a single image, provide the image file
image_path = "path/to/your/image.jpg"

nsfw_probability = n2.predict_image(image_path)

print(f"NSFW probability: {nsfw_probability}")

# Define a threshold for blocking uploads
THRESHOLD = 0.5
if nsfw_probability > THRESHOLD:
    print("Content is likely NSFW and should be blocked.")
else:
    print("Content is likely SFW.")
```





YouTube

James Briggs · 9:23



NSFW



Multi-Modal NSFW Detection with AI

Watch >

Uploaded: Mar 7, 2024 · 52 Likes

Using multi-modal models like OpenAI's CLIP we can use the Semantic Router library for detection of specifi... [More](#) ▾

Source: <https://www.youtube.com/watch?v=EqKjaLrpeI4>

10.2 Moderation Pipeline

- Basic flow:
 - upload
 - safety scan
 - allow / block / review
 - only then continue to OCR or VLM
- Confidence thresholds determine policy routing.
- Some images should be rejected; some should be escalated.
- Safety is a system policy layer, not just model output.